Disaster Response Resource Coordinator

Software Requirements & Feature Specification

Team size: 3 **Duration:** ~1 semester (12 weeks) **Stack:** Full-stack web + agentic AI

---

1. Overview

An AI-driven command-and-control platform for disaster response teams. It ingests incident reports from multiple sources, uses a multi-agent AI system to triage and prioritize them, and actively guides the dispatch and routing of physical resources (ambulances, supply trucks, personnel) to incidents — with a human commander able to review, override, or approve every agent decision.

The primary users are **response coordinators** (command center staff) and **field responders** — not the general public.

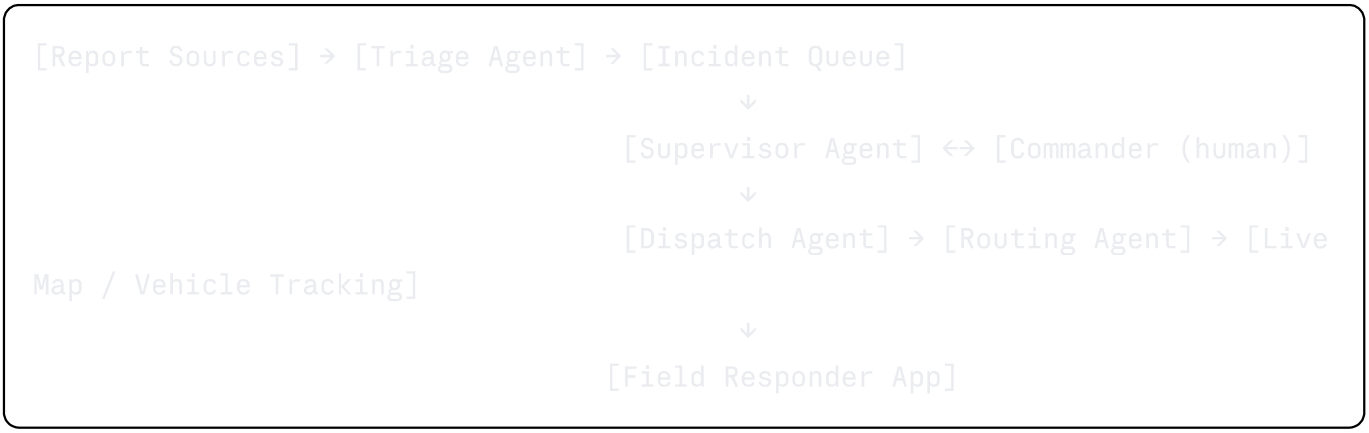
---

2. User Roles

Role	Description	Access
Commander / Coordinator	Oversees the incident queue, approves or overrides agent dispatch decisions, has full map + resource visibility	Full dashboard
Field Responder	Receives assigned incidents, views route, updates status	Mobile-friendly responder view
Report Intake (system)	Ingests external report sources — not a human login	N/A (background service)
Admin	Manages resource inventory (vehicles, personnel, shelters), user accounts	Admin panel

---

3. System Architecture



Backend, agent layer, and frontend communicate over REST (for CRUD) and WebSockets (for live incident/vehicle state).

---

4. Functional Modules

4.1 Report Ingestion

- Accepts reports from multiple input channels: manual form entry (simulating citizen reports), a mock social-media feed (seeded/synthetic data), and a weather/hazard API feed.
- Each report captures: location (lat/lng or address, geocoded), free-text description, optional image, timestamp, source.
- Deduplication: reports describing the same event within a location/time radius are merged into a single incident, with report count shown as a severity signal.

4.2 Triage Agent

- Classifies each incoming report by incident type (flood, fire, structural collapse, medical, etc.) and estimated severity, using NLP on the free-text description.
- Assigns an initial urgency score based on severity, number of corroborating reports, and incident type.
- Flags low-confidence classifications for human review rather than silently guessing.
- Output: a structured incident object added to the Incident Queue.

4.3 Incident Queue & Prioritization

- Central live list of all active incidents, ranked by urgency score.
- Urgency re-computed continuously as new reports arrive or conditions change (e.g., unaddressed incident ages up in priority).
- Commander can manually re-prioritize or merge/split incidents.

4.4 Resource Inventory

- Tracks all dispatchable resources: ambulances, supply trucks, response personnel, shelters.
- Each resource has: type, current location (simulated live GPS), capacity/status (available, en route, on scene, out of service), and capabilities (e.g., "has trauma kit").
- Admin can add/edit/retire resources.

4.5 Dispatch Agent

- For each unresolved incident, evaluates candidate resources by proximity, capability match, and current load.
- Calls the Routing Agent to get real ETA for top candidates rather than assuming nearest = best.
- Produces a ranked recommendation (not a silent auto-assignment) with a plain-language rationale, e.g., "Unit 4 recommended: 3 min closer than Unit 7, carries required equipment."
- On commander approval, resource status updates and a task is pushed to the assigned responder.

4.6 Routing Agent

- Computes actual road-network routes and ETAs (via OSRM or a maps API) for candidate resource–incident pairs.
- Supports multi-stop routing for supply trucks serving several incidents/shelters in one trip.
- Re-routes live if a road is reported blocked or a higher-priority incident interrupts an in-progress route.

4.7 Supervisor Agent (Orchestrator)

- Coordinates the Triage, Dispatch, and Routing agents.
- Detects and resolves conflicts, e.g., two incidents both wanting the same nearest unit.
- Escalates to the human commander whenever confidence is low, stakes are high, or two agent recommendations conflict — the system never auto-dispatches without this human-in-the-loop check for critical actions.
- Maintains a running log of every agent decision and the human response to it, for auditability.

4.8 Command Dashboard (Commander view)

- Live map showing all active incidents and resource positions.
- Incident queue panel with agent-recommended actions, one-click accept/override.
- Resource status panel (available / en route / on scene / out of service).
- Decision log / audit trail.

4.9 Responder View

- Lightweight, mobile-first page for field responders.
- Shows currently assigned incident, route/navigation, and incident details.
- Status update controls: en route → on scene → resolved.
- Status updates push back to the dashboard and Supervisor Agent in real time.

4.10 Vehicle Simulation Layer

- Since real vehicles aren't available, simulates GPS movement of resources along assigned routes over WebSockets, at a configurable speed.
- Enables realistic live-tracking demo without hardware.

4.11 Notifications

- Real-time in-app alerts to commander for new high-severity incidents.
- Real-time push (in-app) to responders when a new incident is assigned.

4.12 Analytics / After-Action View

- Post-incident summary: response time per incident, resource utilization, agent-recommendation acceptance rate (how often the commander agreed with the AI).
- Useful both as a genuine ops feature and as evaluation data for your project report (how good was the agent, really?).

---

5. Non-Functional Requirements

- **Real-time performance:** incident and vehicle state updates should propagate to connected clients within ~1–2 seconds.
- **Explainability:** every agent recommendation must include a human-readable reason, not just a score.
- **Human override:** any agent action affecting real dispatch must be approvable/overridable by a commander; no fully autonomous critical actions.
- **Auditability:** all agent decisions and human responses are logged with timestamps.
- **Geospatial accuracy:** resource-matching queries use proper geospatial indexing (not naive distance loops).

---

6. Tech Stack

Layer	Choice
Frontend	React, Leaflet/Mapbox for maps
Backend	FastAPI (Python) or Node/Express
Agent orchestration	LangGraph or CrewAI, backed by an LLM API
Database	PostgreSQL + PostGIS (geospatial queries)
Real-time	WebSockets
Routing engine	OSRM (self-hosted) or a maps Directions API
Auth	JWT-based, role-based access (commander / responder / admin)

---

7. Data Model (high level)

- **Report** — id, source, text, location, image, timestamp, linked_incident_id
- **Incident** — id, type, severity, urgency_score, status, location, linked reports, created_at
- **Resource** — id, type, current_location, status, capabilities, assigned_incident_id
- **DispatchDecision** — id, incident_id, resource_id, agent_recommendation, human_action (accepted/overridden), rationale, timestamp
- **User** — id, role, name, assigned resource (if responder)

---

8. Suggested Team Split

- **Member A — Ingestion & Triage:** report intake pipeline, NLP classification, deduplication, incident queue logic.
- **Member B — Agent Orchestration & Dispatch:** Supervisor/Dispatch/Routing agents, tool-calling, conflict resolution, routing integration.
- **Member C — Real-time Systems & Frontend:** WebSocket infrastructure, vehicle simulation, command dashboard, responder view.

---

9. Evaluation / Demo Plan

- Seed a simulated disaster scenario (e.g., flood across a mock city) with a burst of incoming reports.
- Show the Triage Agent classifying and queuing incidents live.

- Show the Dispatch Agent recommending resource assignments with rationale, commander approving/overriding.
- Show live vehicle movement and status updates from the responder view.
- Present the after-action analytics: response times, agent acceptance rate.